



Modélisation et gestion d'une base MongoDB pour LearnHub

sujet : Conception et manipulation d'une base de données pour une plateforme d'apprentissage en ligne.

Réalise par : Touzani Mohamed

Filière : B3 cybersécurité , 2025-2026

Date : Le 03/01/2026

INTRODUCTION GÉNÉRALE

Le projet LearnHub consiste à concevoir une base de données MongoDB pour une plateforme de gestion de cours en ligne, à l'exposer via une API REST en Python Flask, et à créer une interface web en HTML, CSS et JavaScript.

Le projet est divisé en cinq parties progressives :

- Partie 1 — Création de la base de données avec les 5 collections et insertion des données initiales.
- Partie 2 — Démonstration de la maîtrise des opérations MongoDB via 30 requêtes (`queries.mongosh.js`).
- Partie 3 — Exposition des données via 7 endpoints d'une API REST Python Flask testés avec Postman.
- Partie 4 — Implémentation de 5 endpoints de logique métier complexe avec opérations enchaînées.
- Partie 5 — Fonctionnalités bonus : recherche textuelle, statistiques, export JSON et interface HTML.

PARTIE 1 — CRÉATION DE LA BASE DE DONNÉES

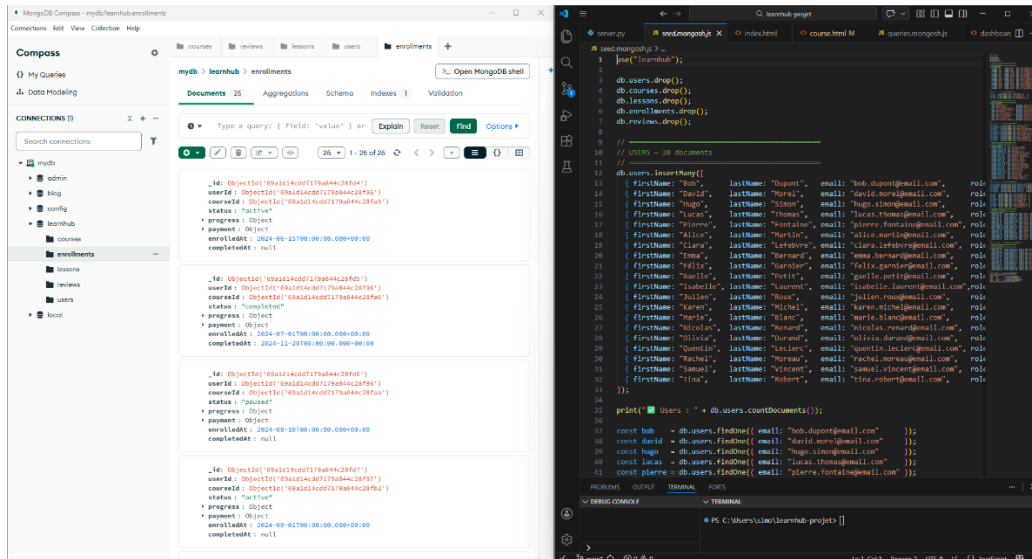
Cette partie consiste à créer la base de données learnhub dans MongoDB et à la peupler via le fichier `seed.mongosh.js`. Ces données servent de fondation pour toutes les phases suivantes du projet.

Les 5 Collections

- `users` — 20 utilisateurs (étudiants et instructeurs) avec profil, compétences et statistiques.
- `courses` — 15 cours avec catégorie, difficulté, prix, tags et métadonnées.
- `lessons` — 30 leçons rattachées aux cours avec type (vidéo/texte) et durée.
- `enrollments` — 25 inscriptions avec progression, statut et informations de paiement.
- `reviews` — 20 avis avec note et commentaire liés à un cours et un utilisateur.

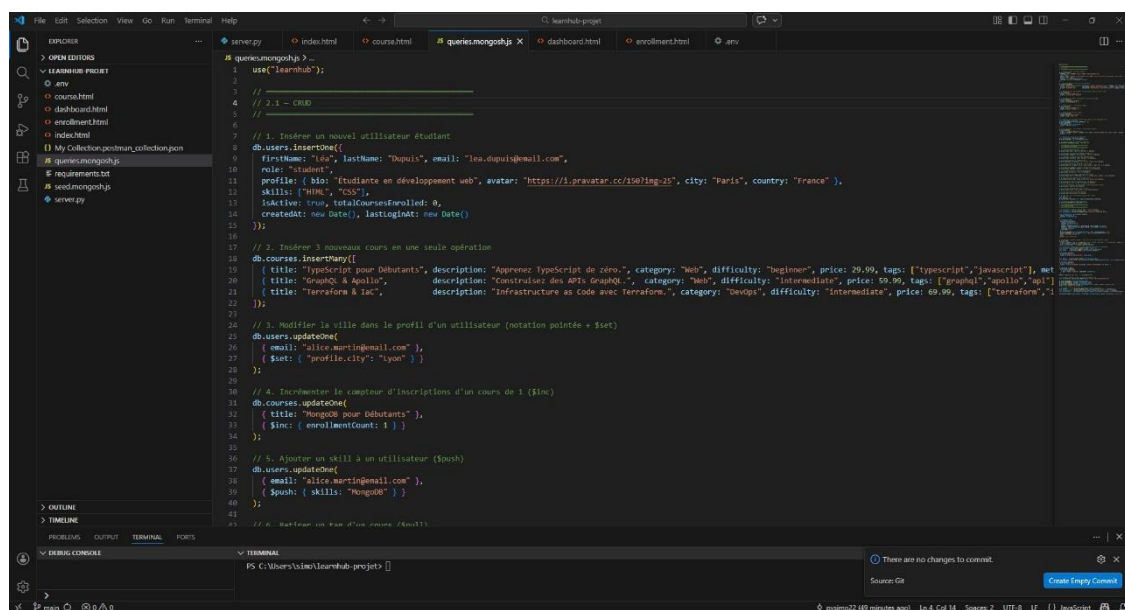
Fichier seed.mongosh.js

Le script utilise insertOne et insertMany pour insérer les documents dans chaque collection après avoir sélectionné la base learnhub avec use('learnhub').



Visualisation dans MongoDB Compass

MongoDB Compass confirme l'insertion réussie des données dans les 5 collections de la base learnhub.

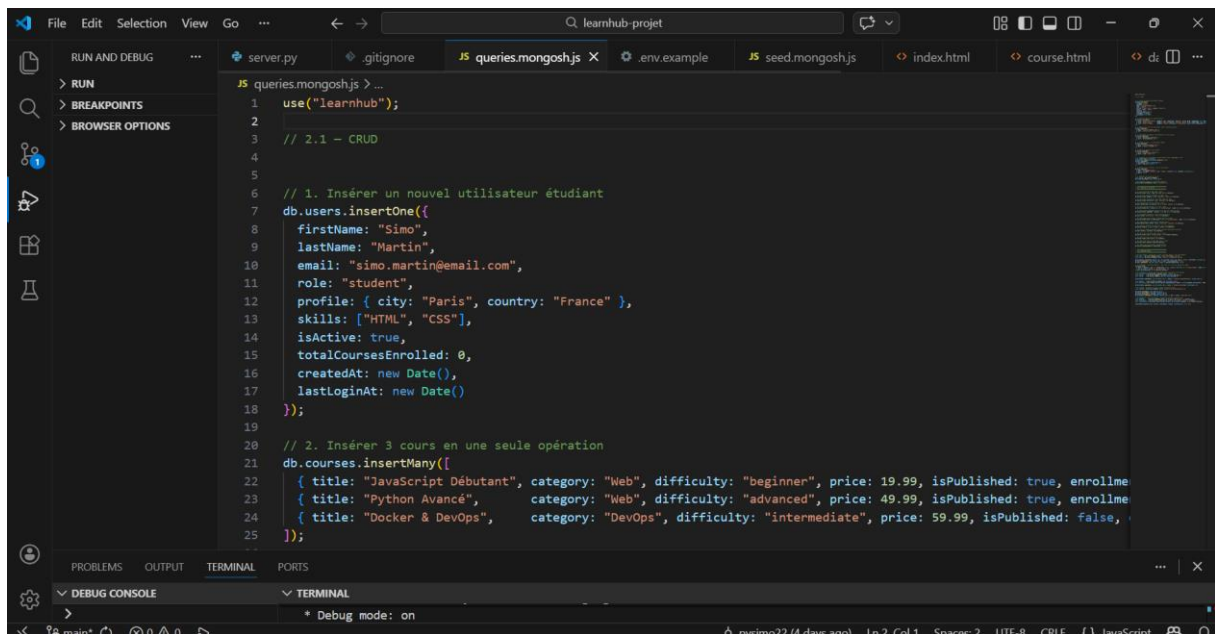


PARTIE 2 — REQUÊTES MONGODB

Le fichier queries.mongosh.js contient 30 requêtes MongoDB qui démontrent la maîtrise des opérations CRUD et des opérateurs avancés sur toutes les collections de learnhub.

Opérations couvertes

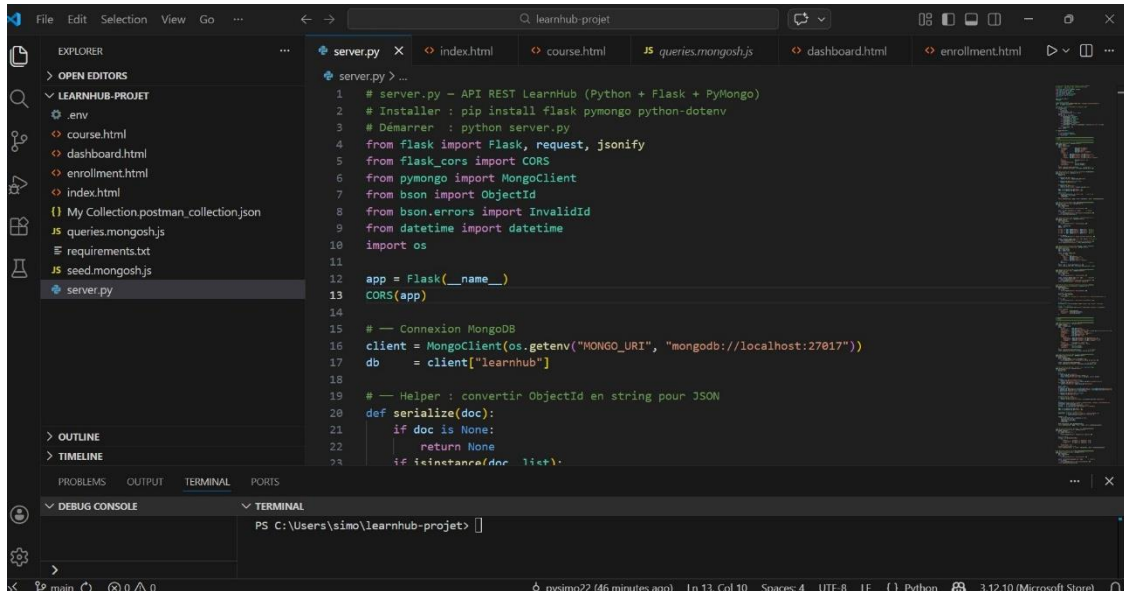
- CRUD de base : insertOne, insertMany, find, findOne, updateOne, deleteOne, deleteMany.
- Opérateurs de mise à jour : \$set, \$push, \$pull, \$inc, \$unset avec notation pointée.
- Opérateurs de requête : \$or, \$in, \$regex, \$gte, \$lte, \$ne.
- Requêtes avancées : trier, limite, countDocuments.



```
1 use("learnhub");
2
3 // 2.1 - CRUD
4
5
6 // 1. Insérer un nouvel utilisateur étudiant
7 db.users.insertOne({
8   firstName: "Simo",
9   lastName: "Martin",
10  email: "simo.martin@email.com",
11  role: "student",
12  profile: { city: "Paris", country: "France" },
13  skills: ["HTML", "CSS"],
14  isActive: true,
15  totalCoursesEnrolled: 0,
16  createdAt: new Date(),
17  lastLoginAt: new Date()
18 });
19
20 // 2. Insérer 3 cours en une seule opération
21 db.courses.insertMany([
22   { title: "JavaScript Débutant", category: "Web", difficulty: "beginner", price: 19.99, isPublished: true, enrollme
23   { title: "Python Avancé", category: "Web", difficulty: "advanced", price: 49.99, isPublished: true, enrollme
24   { title: "Docker & DevOps", category: "DevOps", difficulty: "intermediate", price: 59.99, isPublished: false,
25 ]]);
```

PARTIE 3 — API REST

L'API REST a été développée en Python Flask avec PyMongo. Le fichier `server.py` définit tous les endpoints HTTP. Flask-CORS est activé pour autoriser les requêtes depuis le frontend HTML.

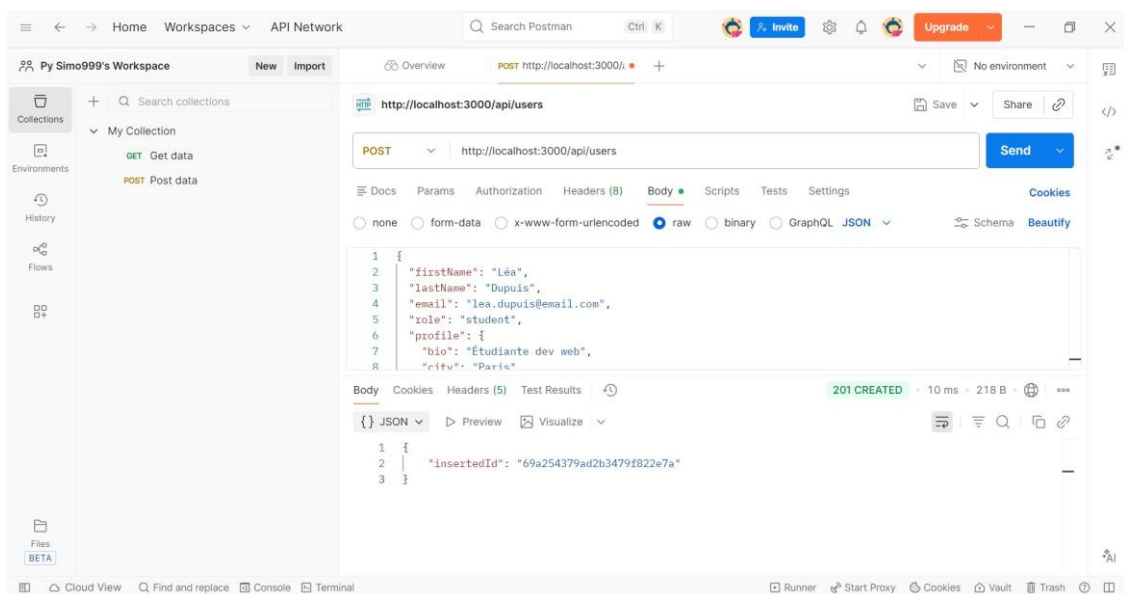


```
1 # server.py - API REST LearnHub (Python + Flask + PyMongo)
2 # Installer : pip install flask pymongo python-dotenv
3 # Démarrer : python server.py
4 from flask import Flask, request, jsonify
5 from flask_cors import CORS
6 from pymongo import MongoClient
7 from bson import ObjectId
8 from bson.errors import InvalidId
9 from datetime import datetime
10 import os
11
12 app = Flask(__name__)
13 CORS(app)
14
15 # --- Connexion MongoDB
16 client = MongoClient(os.getenv("MONGO_URI", "mongodb://localhost:27017"))
17 db = client["learnhub"]
18
19 # --- Helper : convertir ObjectId en string pour JSON
20 def serialize(doc):
21     if doc is None:
22         return None
23     if isinstance(doc, ObjectId):
24         return str(doc)
```

Endpoint 1 — Création d'un utilisateur

Crée un nouvel utilisateur dans la collection `users` via `insertOne`. La réponse retourne l'identifiant MongoDB généré automatiquement (`insertedId`). Statut : 201 CREATED.

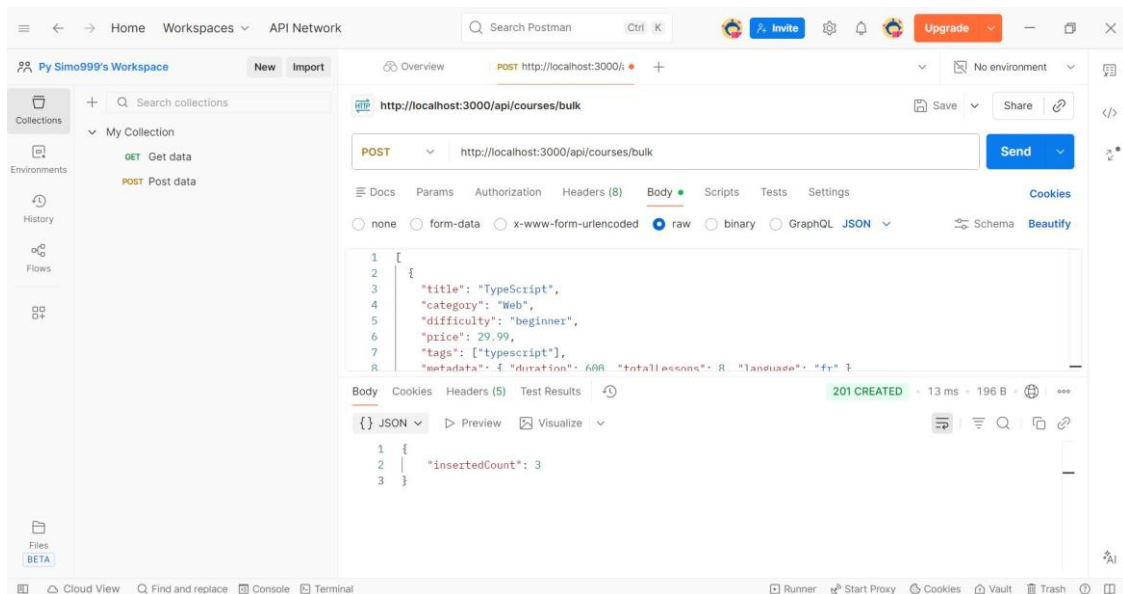
POST `/api/users`



Endpoint 2 — Insertion de cours en masse

Insère plusieurs cours simultanément via insertMany. La réponse confirme le nombre de documents insérés (insertedCount: 3). Statut : 201 CREATED.

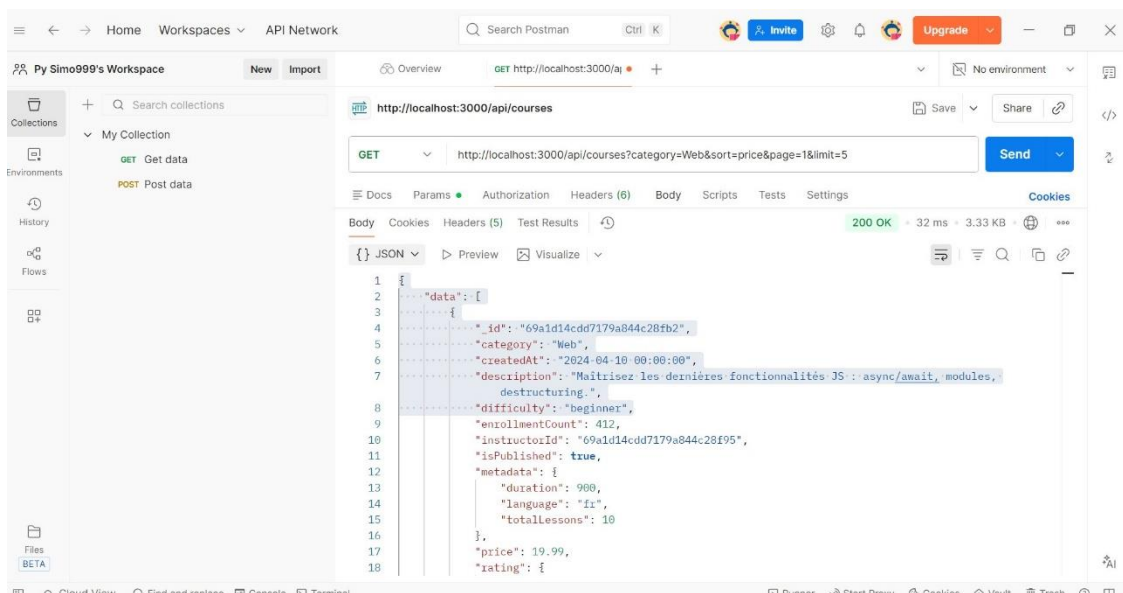
POST /api/courses/bulk



Endpoint 3 — Liste des cours avec filtres

Récupère les cours avec filtrage par catégorie, tri par prix et pagination via `find().sort().skip().limit()`. Statut : 200 OK.

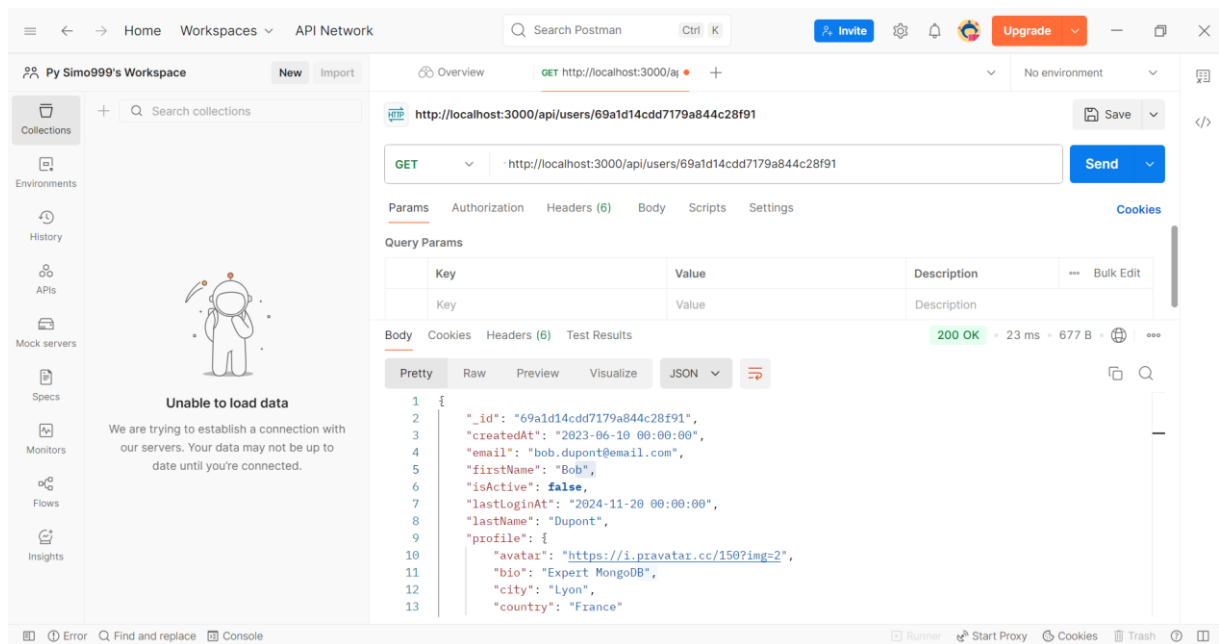
GET /api/courses?category=Web&sort=price&page=1&limit=5



Endpoint 4 — Récupération d'un utilisateur

Récupère le profil complet d'un utilisateur par son identifiant MongoDB via `findOne()`. Statut : 200 OK.

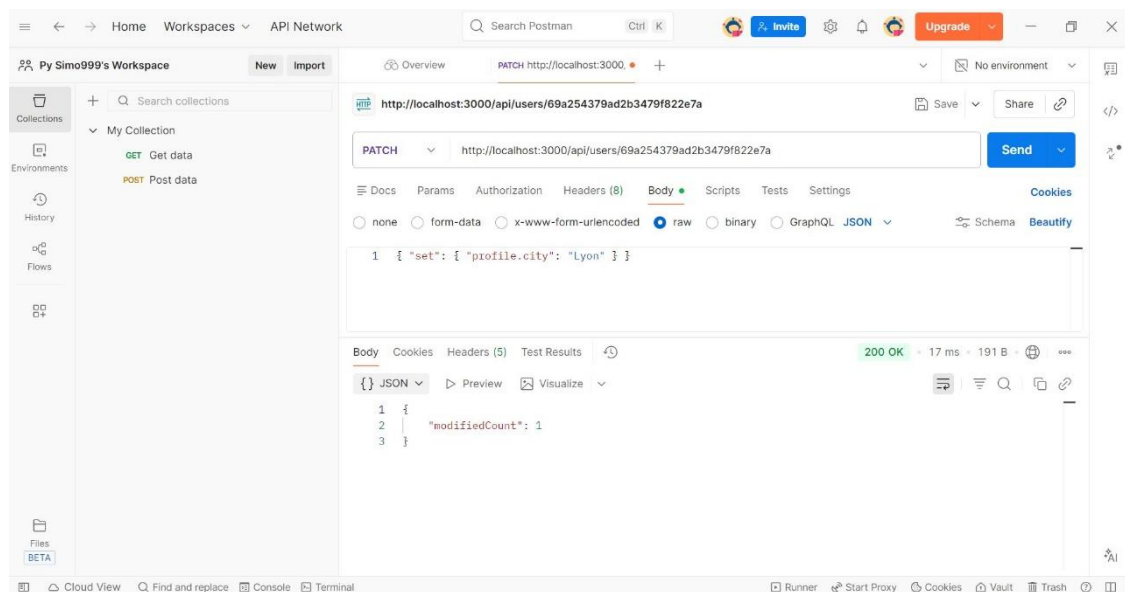
GET /api/users/:id



Endpoint 5 — Mise à jour partielle

Modifie un champ précis avec l'opérateur \$set et la notation pointée (profile.city) sans toucher aux autres champs. Statut : 200 OK.

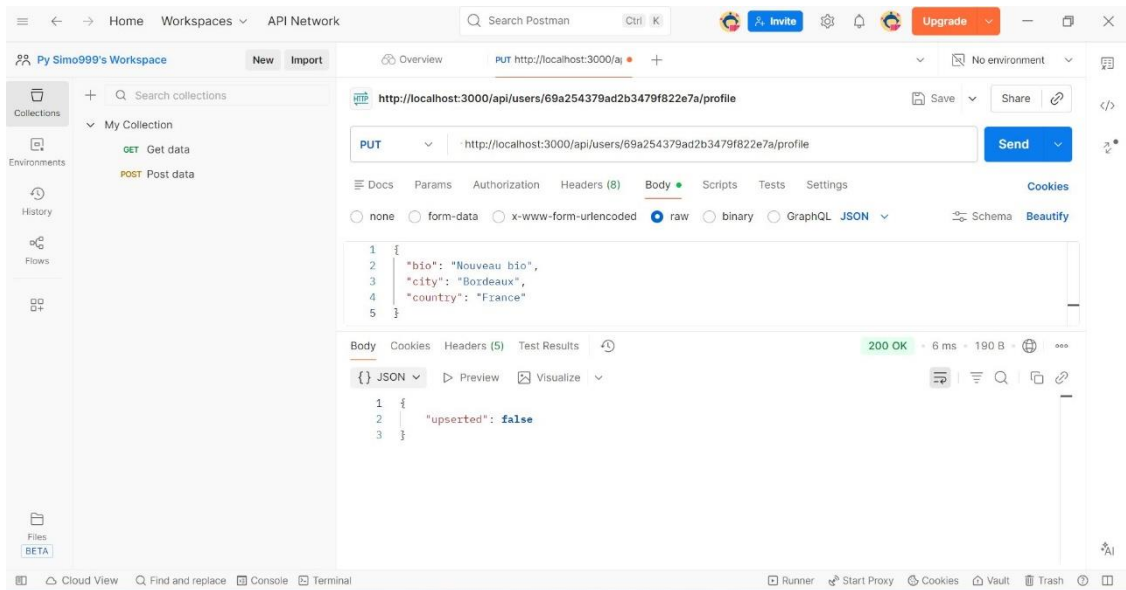
PATCH /api/users/:id



Endpoint 6 — Upsert du profil

Met à jour le profil avec l'option upsert : crée le document s'il n'existe pas, le modifie s'il existe déjà. Statut : 200 OK.

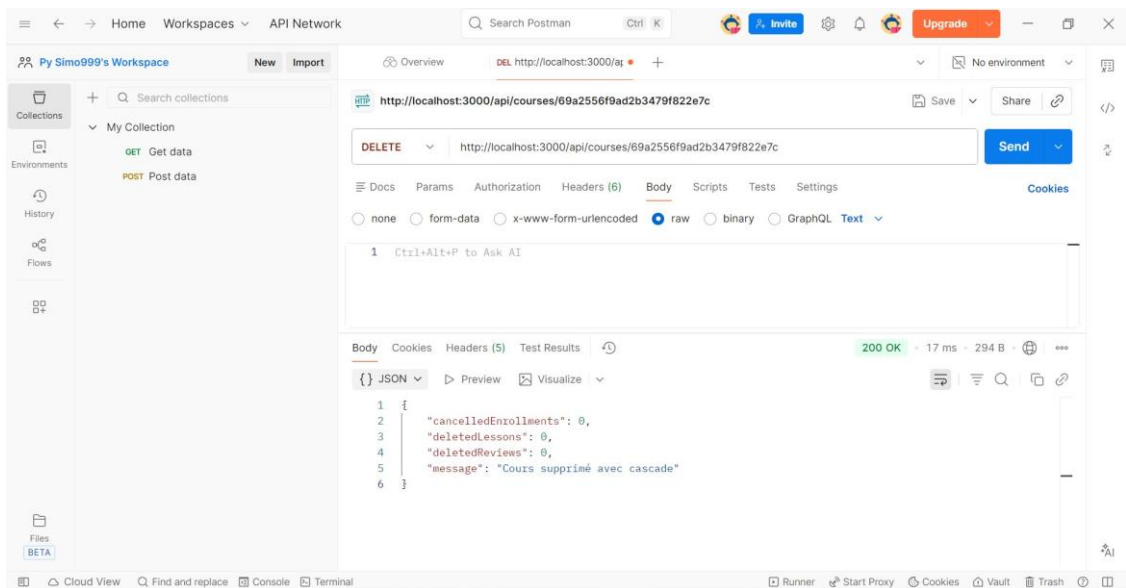
PUT /api/users/:id/profile



Endpoint 7 — Suppression en cascade

Supprime un cours et toutes ses données associées : leçons, avis et inscriptions annulées. Enchaîne `deleteOne`, `deleteMany` et `updateMany`. Statut : 200 OK.

DELETE /api/courses/:id



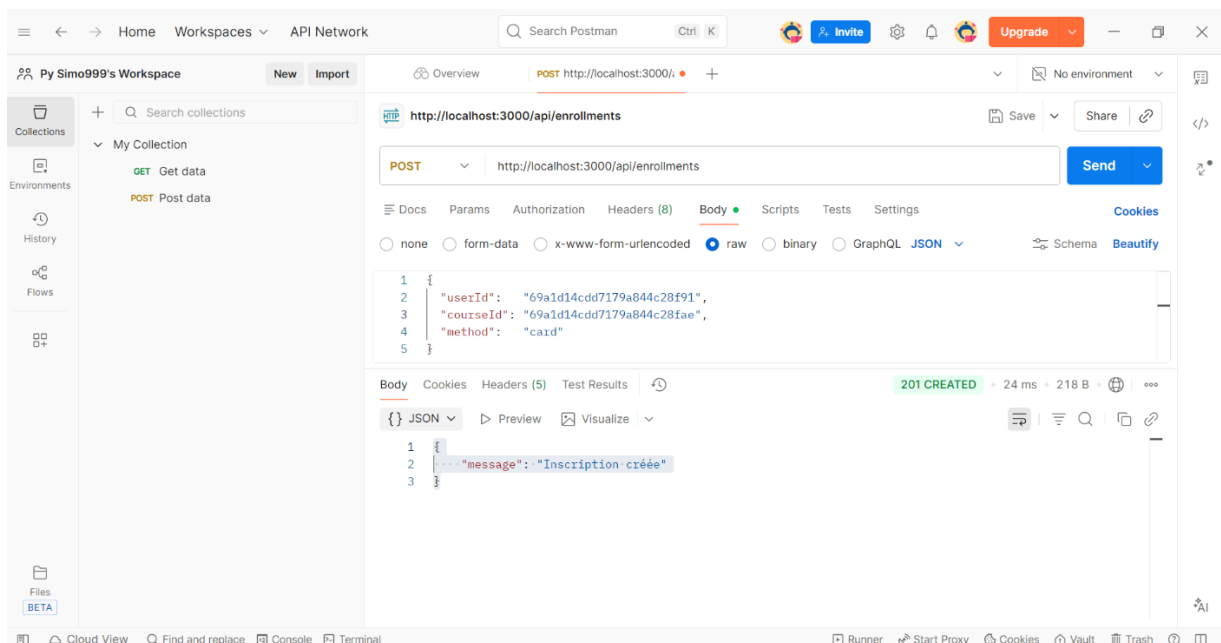
PARTIE 4 — LOGIQUE MÉTIER

Cette partie implémente 5 endpoints complexes qui orchestrent plusieurs opérations MongoDB en séquence avec des validations croisées entre collections pour garantir la cohérence des données.

Endpoint 1 — Inscription à un cours

Vérifie l'existence de l'étudiant et du cours, contrôle l'absence de doublon, crée l'inscription et incrémente les compteurs via \$inc. Six opérations MongoDB enchaînées. Statut : 201 CREATED.

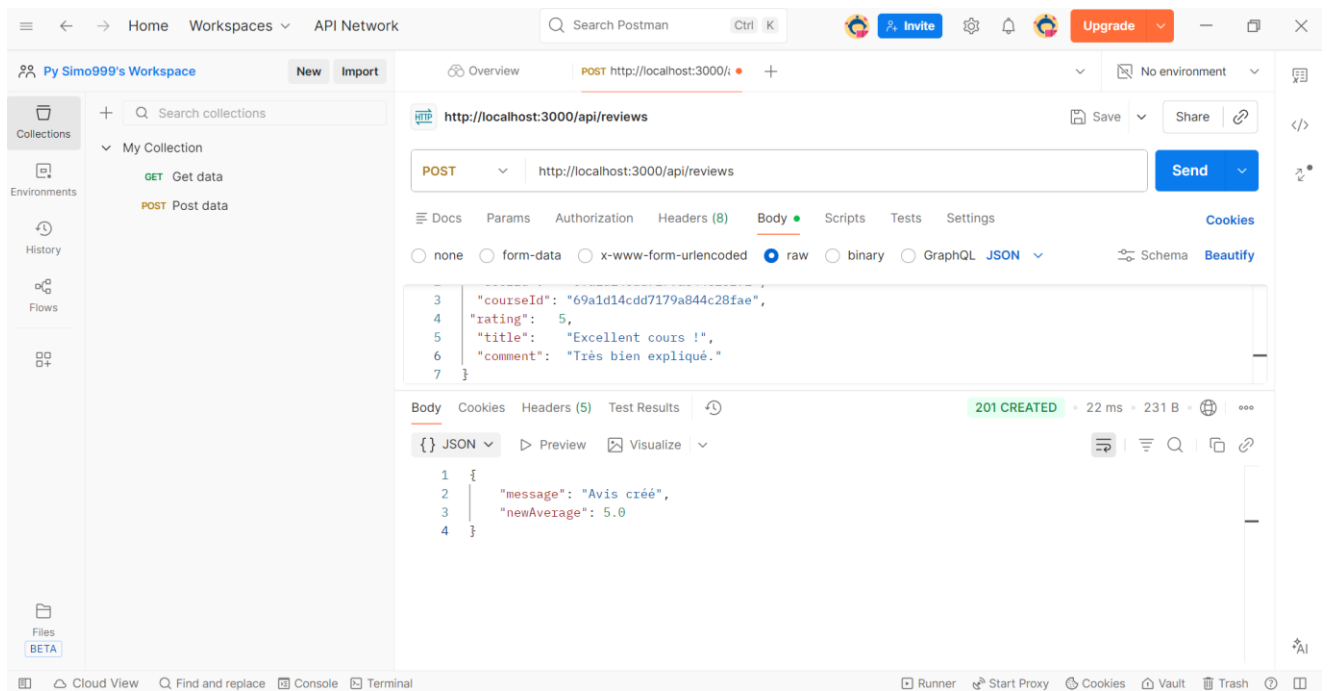
POST /api/enrollments



Endpoint 2 — Soumission d'un avis

Vérifie que l'étudiant est inscrit, enregistre l'avis, puis recalcule automatiquement la note moyenne du cours. La réponse retourne la nouvelle moyenne (newAverage). Statut : 201 CREATED.

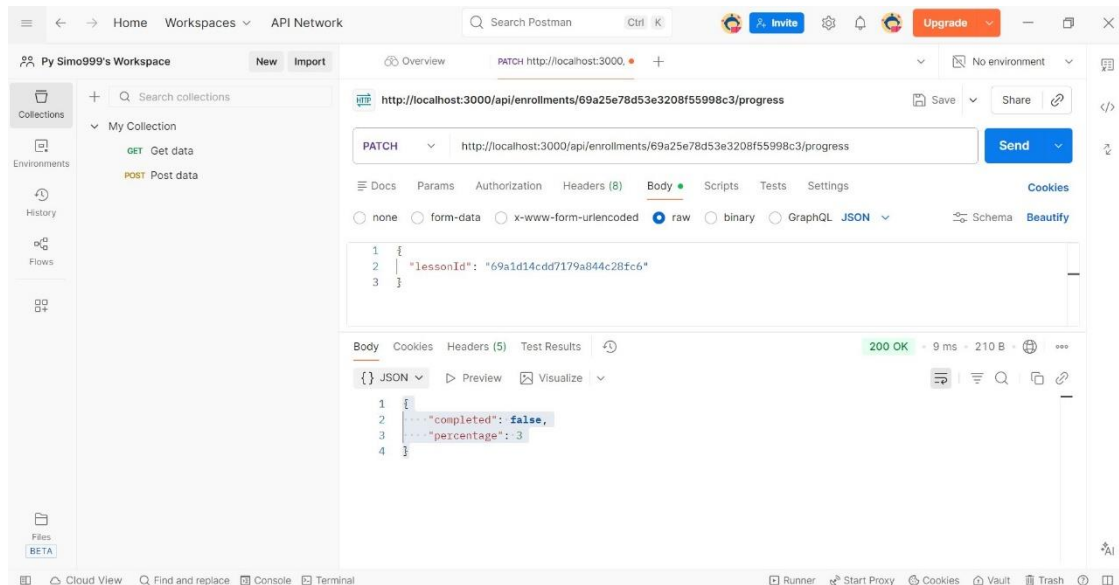
POST /api/reviews



Endpoint 3 — Progression dans un cours

Marque une leçon comme complétée via \$push et recalcule le pourcentage de progression via \$set. Si 100% est atteint, le statut passe automatiquement à completed. Statut : 200 OK.

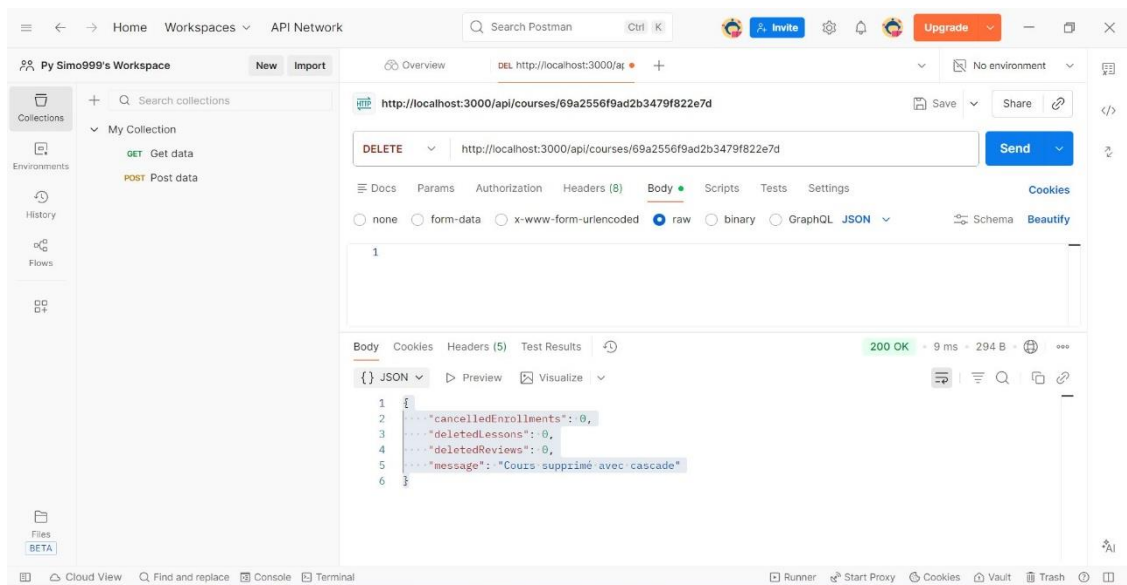
PATCH /api/enrollments/:id/progress



Endpoint 4 — Suppression en cascade

Suppression complète d'un cours avec détail du bilan : deletedLessons, deletedReviews et cancelledEnrollments. Statut : 200 OK.

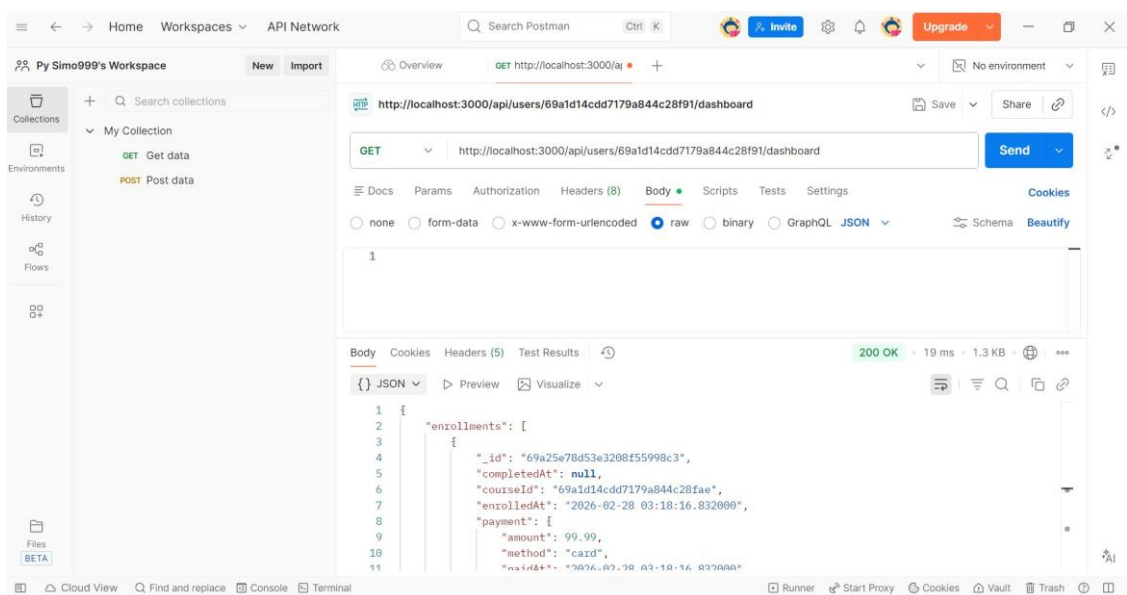
DELETE /api/courses/:id



Endpoint 5 — Dashboard utilisateur

Agrège les données de 3 collections en un seul appel API : profil utilisateur, inscriptions actives et avis soumis. Statut : 200 OK.

GET `/api/users/:id/dashboard`



PARTIE 5 — FONCTIONNALITÉS BONUS

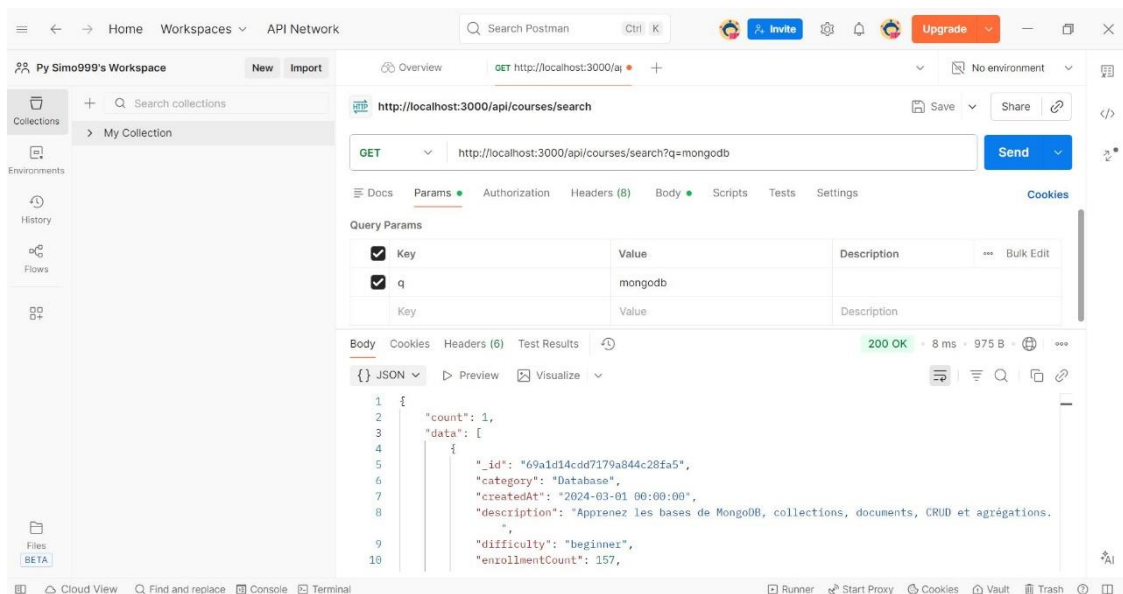
Des fonctionnalités avancées supplémentaires ont été développées pour enrichir l'API, accompagnées d'une interface web HTML complète connectée à l'API REST.

Bonus API

Recherche textuelle

Recherche un mot-clé dans les titres, descriptions et tags des cours via l'opérateur \$regex combiné avec \$or. Insensible à la casse grâce à l'option \$options: 'i'.

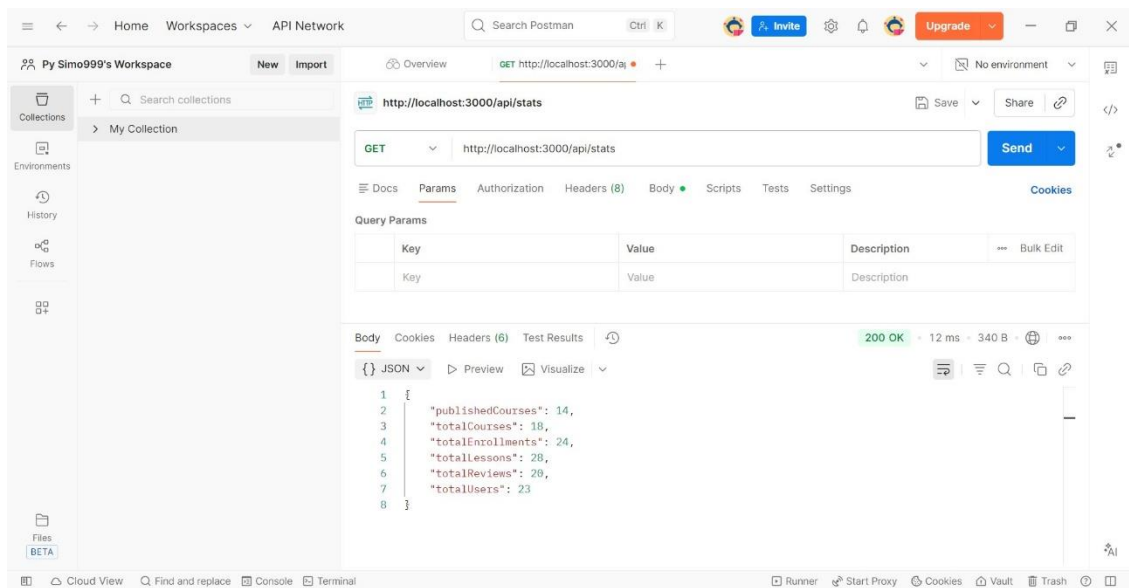
GET /api/courses/search?q=mongodb



Statistiques globales

Retourne en temps réel le total des documents de chaque collection via countDocuments(). Utile pour un tableau de bord d'administration de la plateforme.

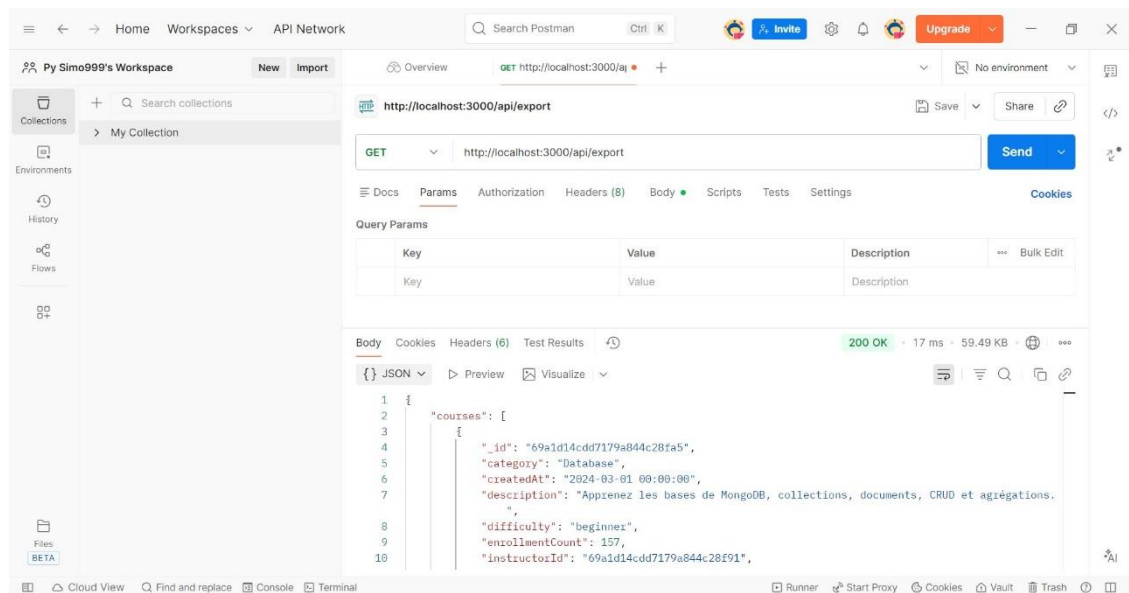
GET /api/stats



Export JSON

Exporte l'intégralité de la base de données en JSON via find() sans filtre sur les 5 collections.

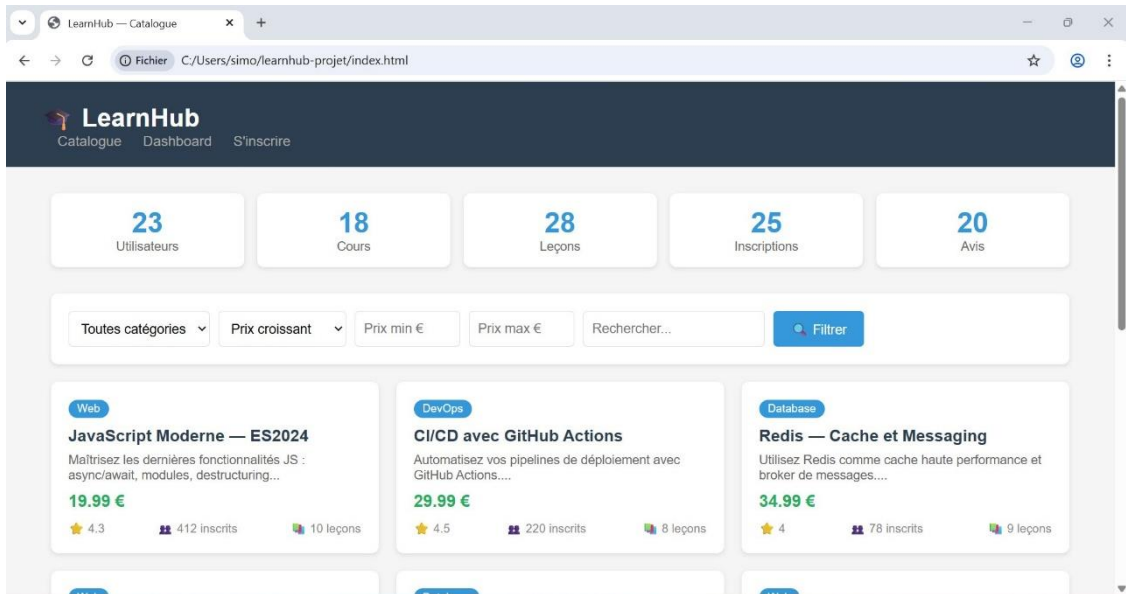
GET /api/export



Bonus Frontend HTML

Interface web développée en HTML5, CSS3 et JavaScript Vanilla. Le CSS est intégré directement dans les fichiers HTML. Le JavaScript utilise fetch() pour appeler l'API et met à jour le DOM dynamiquement.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>LearnHub - Catalogue</title>
6 </head>
7 <body>
8   <div>
9     <div>
10       <div>
11         <div>
12           <div>
13             <div>
14               <div>
15                 <div>
16                   <div>
17                     <div>
18                       <div>
19                         <div>
20                           <div>
21                             <div>
22                               <div>
23                                 <div>
```



CONCLUSION

Le projet LearnHub a été réalisé avec succès en couvrant l'ensemble des cinq parties demandées. La base de données MongoDB a été modélisée avec 5 collections et peuplée de données réalistes via `seed.mongosh.js`.

Les 30 requêtes MongoDB de `queries.mongosh.js` démontrent la maîtrise des opérateurs CRUD et avancés. L'API REST expose des endpoints fonctionnels testés dans Postman, et la logique métier garantit l'intégrité des données grâce aux validations croisées et aux opérations en cascade.

Les fonctionnalités bonus et le frontend HTML connecté à l'API complètent le projet et démontrent une maîtrise solide de MongoDB et de Python Flask.
